

Distance / Fuel-cost Route Planner

Scott Morrison – 190014000

20th May 2026

Section 1: Program Overview

This project implements a C++ route-planning system for a small Scottish goods-transport company. At startup the program loads 10 cities and 15 roads from graph.txt. The user enters a start, destination, and either picks one of three lorries or selects "compare all three". The output lists the route, every refuelling stops with litres and cost, the total distance, and the total fuel cost. Both optional extras are implemented: continue-journey using the remaining fuel, and auto-recommendation of the cheapest lorry.

Section 2: Class Structure

Class/file	Purpose	Key Methods
Graph (graph.h, graph.cpp)	Stores the road network as an adjacency list. It loads data from text file. Will run Dijkstra's algorithm.	LoadData, getShortestPath, getCities, getAdj, findCityIndex, hasCity, printAvailableCities
Vehicle (vehicle.h, vehicle.cpp)	Models a lorry (tank size, miles per litre, name). Simulates a journey along a given path with refuelling logic.	SimulateJourney, getName, getTankCapacity, getMpl
Main.cpp	Main console user interface which contains menus, input handling, integration of Graph + Vehicle files.	main
Tests.cpp	Independent test driver with 9 assertions covering graph parsing, pathfinding, and cost calculation.	RunTests, lambda check

The code is split into two classes, Graph and Vehicle with main.cpp providing the user interface and tests.cpp providing an independent test driver. Both classes use private data members with read-only accessors for encapsulation. A class diagram showing the dependencies is given in Appendix A.3 (Figure 2).

Section 3: Data Structures

3.1 Adjacency list (`std::vector<std::vector<std::pair<int, double>>>`)

The road network is stored as an adjacency list (vector of vectors of pairs), where each entry holds a neighbour's index and the road distance. With 10 nodes and 15 edges the graph is sparse, so an adjacency matrix would waste 100 cells. The list uses $O(V+E)$ memory and gives $O(\deg(v))$ neighbour

iteration, exactly the access pattern Dijkstra uses. The abstract concept is illustrated in Appendix A.2, (Figure 1)

3.2 `std::map<std::string, int>` for name -> index lookup

User-typed city names are mapped to integer indices through `std::map` ($O(\log N)$ lookups, sorted order). `Std::unordered_map` would offer $O(1)$ average lookups but no defined order; for 10 cities the difference is negligible, so I chose the sorted map for predictability.

3.3 `std::priority_queue` (min-heap) for Dijkstra's frontier.

Dijkstra's algorithm repeatedly removes the lowest-cost unvisited node from a frontier; a min-heap via `std::priority_queue` does this in $O(\log V)$ per pop. A linear scan would be $O(V)$ per pop — fine for 10 nodes but the heap scales properly.

Section 4: Algorithm choice

4.1 Dijkstra's algorithm for shortest path.

I selected Dijkstra's algorithm because all edge weights are positive, guaranteeing the optimal solution in $O((V+E) \log V)$. I considered two alternatives: breadth-first search finds shortest paths by hop count but ignores edge weights, so it would return wrong answers on a weighted map; Bellman-Ford handles negative edge weights at $O(VE)$ cost, which is unnecessary here. Dijkstra is the standard choice for non-negative weighted shortest-path problems. A worked example of Dijkstra's algorithm running on a small graph is shown in Appendix A.4 (Figure 3).

4.2 Cost vs. Distance

In general, the cheapest cost and shortest distance routes can differ, a longer route through cheaper fuel cities can be cheaper overall. However, all three lorries have tank ranges greater than the longest plausible journey on this map (80L x 12 mpl = 960 miles), so every journey fits in one fill at origin. Under that condition the total cost depends only on tank size and origin price, so the shortest distance route is also the cheapest. Keeping logic in the Vehicle class means a larger map or smaller tank could be supported later without changing the routing code.

Section 5: Test Results

A separate tests.cpp runner uses a `check(name, condition)` lambda and runs 9 assertions against the live data file. All pass under `-Wall -Wextra` with zero warnings. The full list of tests is shown in Appendix A.1 (Table 1).

Section 6: Build & Run

Compiles cleanly under g++ with `-std=c++17 -Wall -Wextra`. Build: `g++ -std=c++17 -Wall -Wextra main.cpp graph.cpp vehicle.cpp -o planner.exe` Then run `./planner.exe` from the same folder (graph.txt must be present). Tests build the same way with tests.cpp in place of main.cpp.

Section 7 AI Acknowledgement:

In line with the module's Amber AI-usage rating, I declare that Anthropic's Claude and Google's Gemini was used as a study aid during this coursework. The tools were used to discuss algorithm and data-structure trade-offs, clarifying C++ syntax, and help debug compile errors. All submitted source code was written by me, reviewed line by line, and tested against the assignment requirements; the design decisions and final wording of this report are my own.

Appendix A: Test Results and Diagrams

A.1 - Table 1: Automated Test Results

What is Tested?	Result
Graph loads exactly 10 cities	PASS
Wick's fuel price is GBP 2.90	PASS
"Dundee" is in the lookup table	PASS
"Atlantis" is <i>not</i> in the lookup table	PASS
Dundee -> Glasgow shortest path has 3 cities	PASS
Dundee -> Glasgow total distance is 84 miles	PASS
Small Lorry Dundee -> Glasgow cost is GBP 148.00	PASS
Fuel remaining after that journey is 73.00 L	PASS
Exactly one refuel stop at Dundee	PASS

A.2 - Figure1: Abstract weighted graph

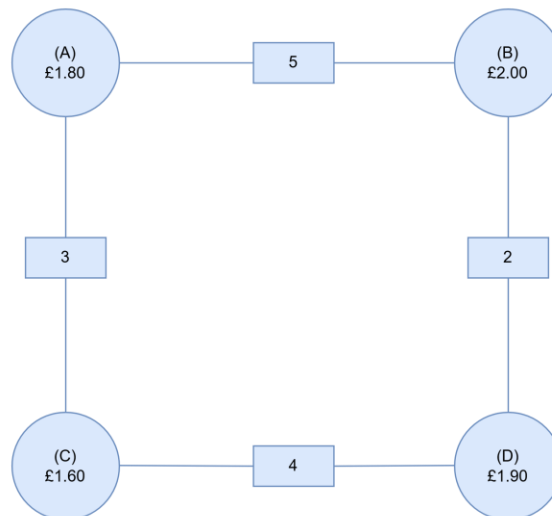


Figure 1: An abstract weighted graph. Circles are nodes (cities) with fuel prices; edges are roads with distance labels.

A.3 - Figure2: Class structure (UML version)

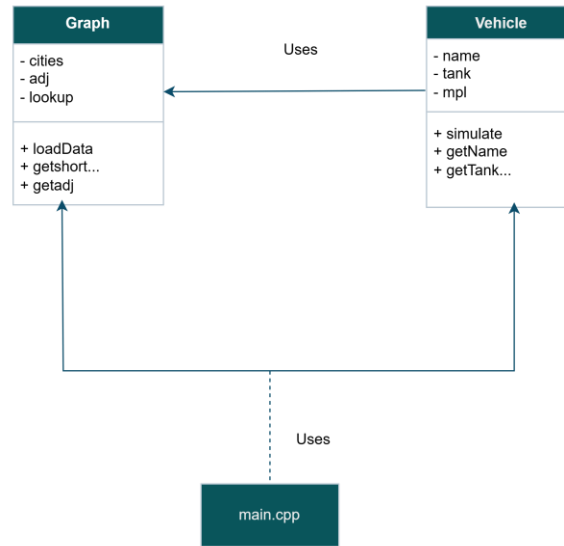


Figure 2: Class structure. Arrows indicate a “uses” relationship. Both classes encapsulate their data privately.

A.4 - Figure 3: Dijkstra's algorithm walkthrough

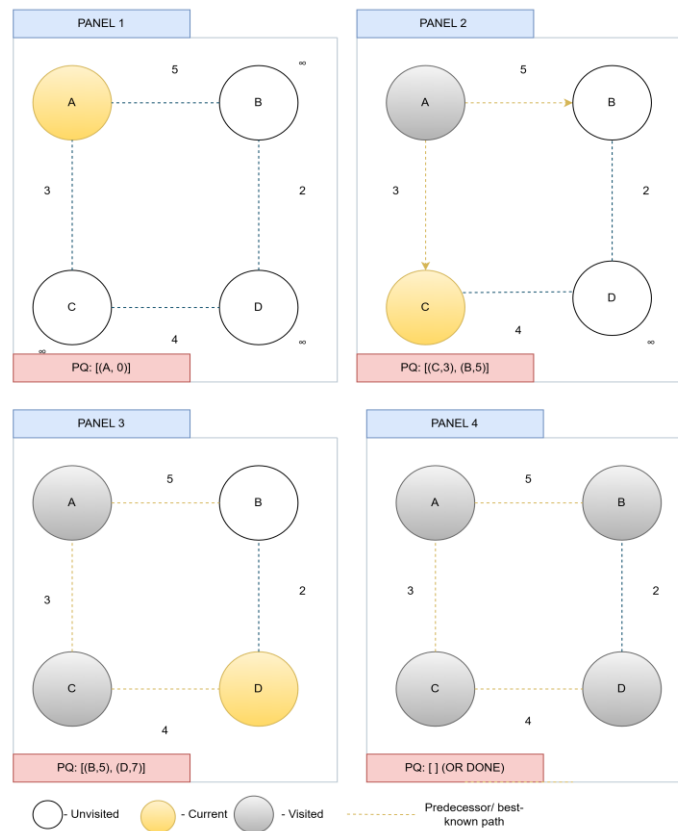


Figure 3: Dijkstra's algorithm running on the abstract graph. Distances are updated whenever a cheaper route to a node is found; the priority queue always pops the lowest-cost unvisited node next.